

TECHNICAL ARCHITECTURE — PHASE 3

Technical Architecture — Phase 3

Full Social Platform

v2.0 · Richard Uzor — May 2026 · Devstrike Digital Limited

01 — PHASE 3 ARCHITECTURAL FOCUS

Phase 3 introduces the most architecturally significant addition: a real-time layer for fellowship rooms, prayer rooms, and direct messaging. This requires WebSocket infrastructure, a message persistence layer for room history, and read replicas for the growing database load.

02 — REAL-TIME ARCHITECTURE

Component	Technology	Notes
WebSocket provider	Ably (managed) or Soketi (self-hosted)	Ably recommended for Phase 3 — managed, reliable, Nigerian PoPs. Soketi for cost optimisation at scale.
Channel model		

Component	Technology	Notes
	One channel per room, one per DM thread, one per live prayer session	Rooms: persistent channels. Live sessions: temporary channels, archived on close.
Message persistence	PostgreSQL via API — all WS messages written to DB	Client sends via WS → server persists to DB → broadcasts to channel subscribers
Presence	Ably Presence API — active member list per room	Used for prayer room 'who's praying now' indicator
Offline messages	On reconnect: client requests missed messages since last_seen timestamp	GET /api/v1/rooms/:id/messages?after=timestamp

03 — NEW API ENDPOINTS (PHASE 3)

Method	Endpoint	Auth	Description
POST	/api/v1/rooms	Account	Create fellowship room
GET	/api/v1/rooms	Account	List rooms — joined, public, recommended
GET	/api/v1/rooms/:id/messages	Member	Room message history (paginated)
POST	/api/v1/rooms/:id/join	Account	Join or request to join room
POST	/api/v1/rooms/:id/members/:userId/warn	Elder	Warn a member
POST	/api/v1/rooms/:id/members/:userId/mute	Elder	Mute a member
POST	/api/v1/prayer	Account	Create prayer request
GET	/api/v1/prayer	Account	List prayer requests in user's rooms
POST	/api/v1/prayer/:id/intercede	Account	Stand in agreement
POST	/api/v1/prayer/:id/answer	Requester	

Method	Endpoint	Auth	Description
			Mark as answered + testimony
POST	/api/v1/prayer/sessions	Elder/ Pastor	Create live prayer session
GET	/api/v1/dm/:userId	Account	Get or create DM thread
GET	/api/v1/dm/:threadId/ messages	Account	DM message history
POST	/api/v1/auth/ws-token	Account	Issue short-lived WebSocket auth token
GET	/api/v1/hq/messages	HQ Role	HQ denomination message monitoring
GET	/api/v1/hq/analytics	HQ Role	Branch analytics dashboard data
POST	/api/v1/subscriptions/ premium	Account	Initiate premium subscription

04 — DATABASE SCALING (PHASE 3)

Concern	Solution
Read load (browse-heavy)	Read replica on Railway. All GET requests routed to replica.
Search (semantic)	pgvector extension. Message embeddings generated on post via OpenAI API. Stored as vector column.
Room message volume	Room messages partitioned by room_id. Older messages archived to cold storage after 12 months.
Connection pooling	PgBouncer enabled on Railway PostgreSQL. Serverless functions safe to scale.
Answered prayer archive	Materialised view refreshed every 15 minutes — aggregates answered prayers with testimony count.

05 — AI SEARCH ARCHITECTURE

1. Indexing

- On message post: API calls OpenAI text-embedding-3-small → returns 1536-dimension vector.
- Vector stored in messages.embedding column (pgvector).
- Index: HNSW index on embedding column for fast approximate nearest-neighbour search.

2. Query

- User types semantic query in search.
- API embeds query string → cosine similarity search against messages.embedding.
- Results ranked by similarity score + recency + reaction count.
- Returns top 20 semantically relevant messages regardless of exact keyword match.